In this example, we employ an agent that uses the Wikipedia search tool available at https://mng.bz/7QN7, which allows the AI to answer a wide range of questions by searching the Wikipedia website in real time. The system now uses a ReAct agent, which can reason about its actions to gather information. The LLM responses are streamed, and the UI is updated in real time to show the agent's thought process, including when it's performing searches and generating responses. Again, we keep the same basic functionality intact: the UI elements stay the same, and we are using the `createStreamableUI` to push UI updates directly to the client as ReAct components.

Project: chat-rsc-langchain-agents

The code for this example project is in the ch06/chat-rsc-langchain-agents folder. You can use that example by running this command in the root folder:

```
$ npm run dev -w ch06/chat-rsc-langchain-agents
```

The appendix provides instructions on how to create a secret key for OpenAI and other providers:

```
// .env

OPENAI_API_KEY=<SECRET_KEY>
GEMINI_API_KEY=<YOUR_GEMINI_API_KEY>
```

Now start the application and ask a question. The agent in the background will try to use the Wikipedia search tool to retrieve information and present this to the user. Since we configured the tool to fetch only a single result, the UI will list the first one found (figure 6.8).

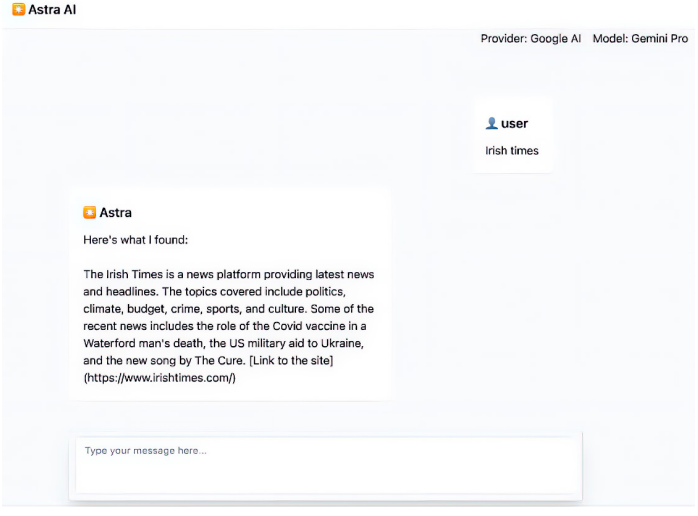


Figure 6.8 Sample interaction with our AI agent that uses `WikipediaQueryRun` to return a search result and display it to the user. Behind the scenes, the agent performs one or more calls to the provided tool to reach its end goals as instructed by the system prompt.